Q. Briefly Explain the History of Database Applications. (7 Marks)

📘 1. Introduction

A Database is an organized collection of data that can be easily stored, accessed, managed, and updated.

Database applications have evolved over time to handle increasing amounts of data efficiently. The history of database applications can be divided into different stages based on the development of storage and data management technologies.

🔹 2. History of Database Applications

1. File-Based Systems (1950s–1960s) 📁

- Initially, data was stored in separate files using file processing systems.
- Each application maintained its own data files.
- Data redundancy and data inconsistency were common problems.
- Sharing and managing data was difficult.

Example: Employee records stored in separate files.

2. Hierarchical Database Model (1960s) 🌳

- Data was organized in a tree-like structure.
- Each child record had only one parent record.
- It supported one-to-many relationships.
- It was used for applications such as banking and airline reservations.

Example: IBM's IMS (Information Management System).

3. Network Database Model (1970s) 🔗

- Data was organized using records and links.
- A record could have multiple parent records.
- It supported many-to-many relationships.
- It was more flexible than the hierarchical model.

Example: CODASYL database model.

Theerthananda Gowda

4. Relational Database Model (1970s–1980s) 📊

- Introduced by Dr. E. F. Codd in 1970.
- Data was stored in tables consisting of rows and columns.
- SQL was developed to manage and retrieve data.
- It reduced data redundancy and made data management easier.

Example: MySQL, Oracle, and SQL Server.

5. Object-Oriented Database Systems (1980s–1990s) 🧩

- Data was stored in the form of objects.
- It supported object-oriented concepts such as classes, objects, and inheritance.
- It was useful for handling complex data.

Example: ObjectDB.

6. Web-Based and Distributed Databases (1990s–2000s) 🌐

- Databases became widely used in web applications and internet services.
- Data could be stored and accessed across multiple locations.
- They supported online transactions and distributed data management.

Example: Online shopping and banking applications.

7. NoSQL and Cloud Databases (2000s–Present) ☁️

- Developed to handle large amounts of structured, semi-structured, and unstructured data.
- They provide high scalability and flexible data storage.
- Cloud databases allow users to access data through the internet.

Example: MongoDB, Cassandra, and Amazon DynamoDB.

Q. Explain the Characteristics of DBMS. (6 Marks)

📘 1. Introduction

DBMS (Database Management System) is software used to create, store, manage, retrieve, and update data in a database.

It provides an efficient way to organize data and allows multiple users to access and manage information securely.

Examples: MySQL, Oracle, PostgreSQL, and SQL Server.

🔹 2. Characteristics of DBMS

1. Data Independence 🔄

- Changes in the database structure do not require major changes to application programs.
- It separates data storage from application programs.

2. Data Security 🔒

- Protects the database from unauthorized access.
- Provides authentication and access permissions to users.

3. Data Integrity ✅

- Maintains the accuracy and correctness of data.
- Uses constraints such as PRIMARY KEY, FOREIGN KEY, and NOT NULL.

4. Data Sharing 🤝

- Allows multiple users to access and share the same database.
- Supports simultaneous access by different users.

5. Reduced Data Redundancy 📉

- Minimizes unnecessary duplication of data.
- Helps maintain consistency throughout the database.

6. Backup and Recovery 💾

Theerthananda Gowda

- Provides facilities to create database backups.
- Helps restore data in case of system failure or accidental data loss.

7. Data Consistency 📊

- Ensures that the same data remains uniform throughout the database.
- Prevents conflicting or incorrect information.

8. Multiple Views of Data 👁️

- Allows different users to view different parts of the same database.
- Each user can access information according to their requirements.

🔹 **3. Example**

Consider a College Management System:

- Students can view their marks.
- Teachers can update student marks.
- Administrators can manage student records.
- The DBMS protects the data and allows authorized users to access it.

Q. Explain the Advantages of Using DBMS Approach. (6 Marks)

📘 **1. Introduction**

DBMS (Database Management System) is software used to store, organize, manage, and retrieve data efficiently.

The DBMS approach provides several advantages over traditional file processing systems by reducing data duplication, improving security, and making data management easier.

🔹 **2. Advantages of Using DBMS Approach**

1. Controls Data Redundancy 📉

- DBMS reduces unnecessary duplication of data.
- It helps avoid storing the same information multiple times.

2. Data Sharing 🤝

- Multiple users can access and share the same database.
- It allows different departments to use common data.

3. Data Security 🔒

- DBMS protects data from unauthorized access.
- It provides user authentication and access permissions.

4. Data Integrity ✅

- DBMS maintains the accuracy and correctness of data.
- Constraints such as PRIMARY KEY and FOREIGN KEY help maintain valid data.

5. Backup and Recovery 💾

- DBMS provides backup facilities to protect important data.
- It helps recover data after system failures or accidental loss.

6. Data Independence 🔄

- Changes in database structure can be made without major changes to application programs.
- It makes database maintenance easier.

7. Concurrency Control 👥

- Allows multiple users to access the database simultaneously.
- Prevents problems when several users try to update the same data.

8. Better Data Management 📊

- DBMS provides easy methods to insert, update, delete, and retrieve data.
- It helps manage large amounts of information efficiently.

🔹 **3. Example**

Consider a College Management System:

- Students can view their marks.

Theerthananda Gowda

- Teachers can update student records.
- Administrators can manage admission details.
- DBMS securely maintains all records in a centralized database.

Q. Explain the Simplified Database System Environment with a Diagram. (6 Marks)

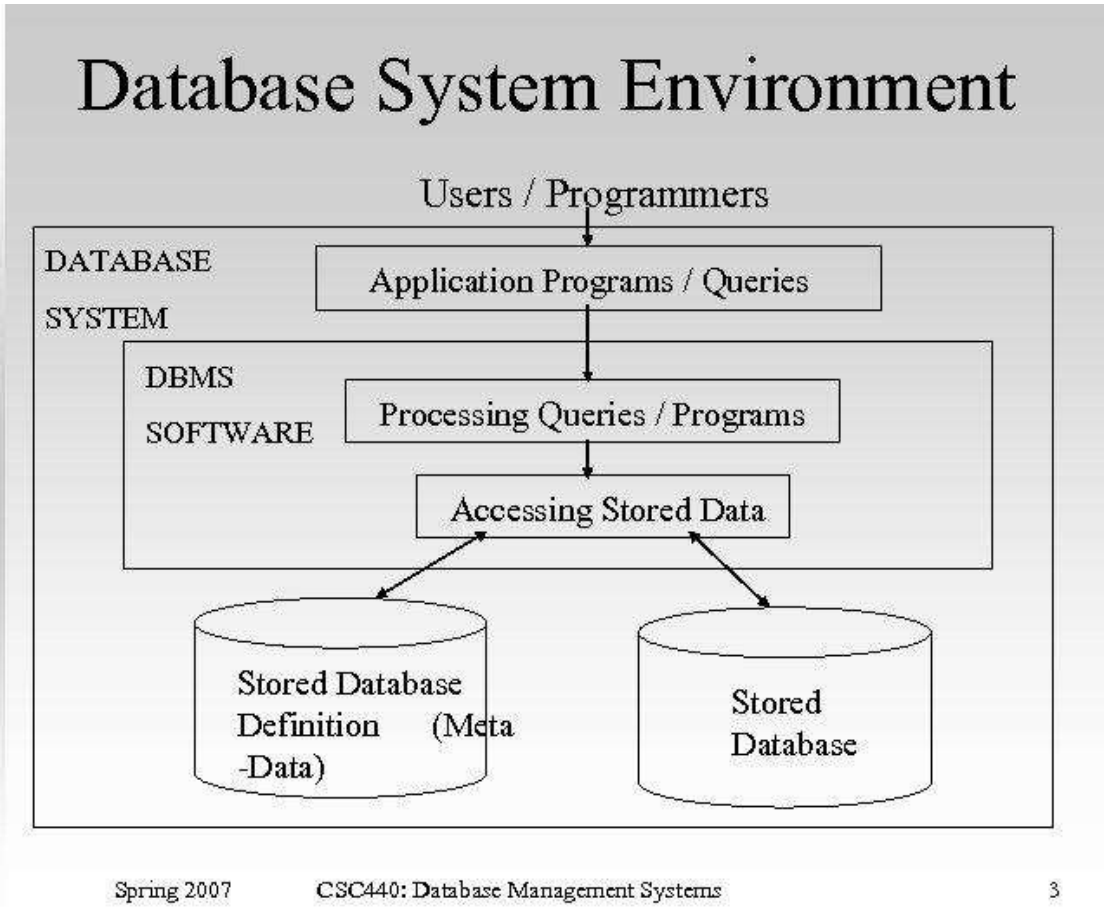
📘 1. Introduction

A Database System Environment is a collection of components that work together to store, manage, retrieve, and update data.

A simplified database system environment shows how users interact with application programs, how the DBMS processes their requests, and how data is stored in the database.

🔹 2. Diagram of Simplified Database System Environment

The following diagram is based on the standard DBMS environment diagram from Fundamentals of Database Systems.



Theerthananda Gowda

🔹 3. Components of Simplified Database System Environment

1. Users / Programmers 👨‍💻

- These are the people who interact with the database system.
- They use application programs or queries to access and manage data.

2. Application Programs / Queries 💻

- These are programs or commands used to communicate with the DBMS.
- They allow users to insert, update, delete, and retrieve data.

3. DBMS Software ⚙️

- DBMS is the main software that manages the database.
- It acts as an interface between users and the stored database.
- It contains two important components:

a) Software to Process Queries / Programs

- Accepts and processes user queries and application programs.
- Checks and prepares queries for execution.

b) Software to Access Stored Data

- Accesses the required data from the database.
- Performs operations such as reading and updating stored data.

4. Stored Database Definition (Metadata) 📝

- Contains information about the database structure.
- Stores table definitions, data types, constraints, and relationships.
- It is also called the data dictionary or database catalog.

5. Stored Database 💾

- Contains the actual data stored in the database.
- Stores information in the form of tables, records, and other database objects.

🔹 4. Working of Database System Environment

The database system works in the following sequence:

1. Users or programmers send requests through application programs or queries.
2. The DBMS receives and processes these requests.
3. The query-processing software checks and prepares the queries.
4. The data-access software retrieves or modifies the required data.
5. The DBMS uses metadata to understand the database structure.
6. The required information is retrieved from or updated in the stored database and returned to the user.

🔹 **5. Example**

Consider a College Management System:

- A student requests to view examination marks.
- The application sends a query to the DBMS.
- The DBMS processes the query and accesses the stored database.
- The student's marks are retrieved and displayed.

Q. Explain Entity Types and Entity Sets. (6 Marks)

📘 **1. Introduction**

In DBMS, the Entity-Relationship (ER) Model is used to represent real-world objects and their relationships in a database.

An Entity is a real-world object that has its own identity and can be identified separately.

Example: Student, Employee, Teacher, and Department.

🔹 **2. Entity Type**

An Entity Type is a collection or category of entities that have the same attributes.

- It defines the common properties of similar entities.
- It describes the structure of an entity.
- It is represented by a rectangle in an ER diagram.

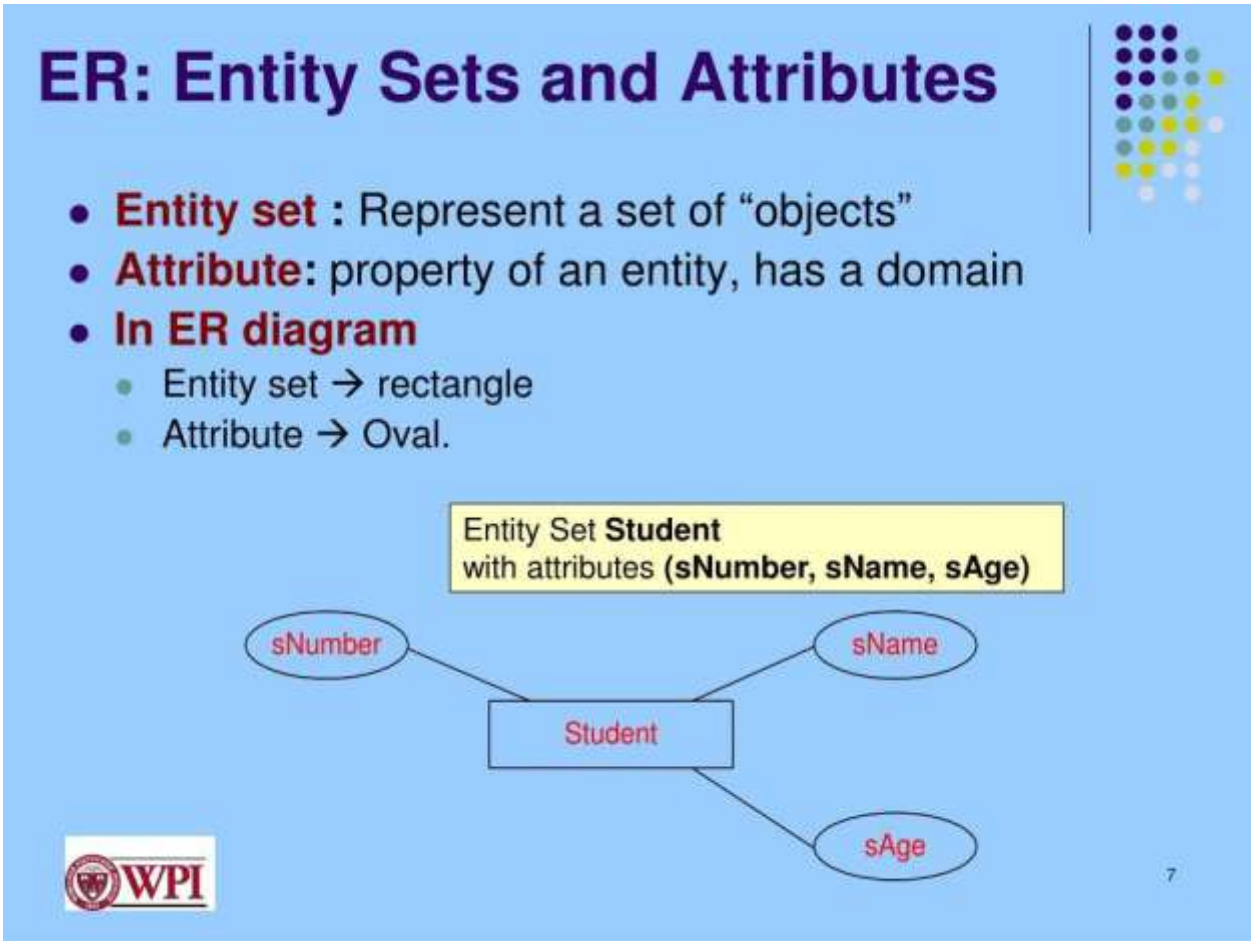
Theerthananda Gowda

- Each entity type has attributes that describe its properties.

Example:

Consider a STUDENT entity type with the following attributes:

- Student_ID
- Student_Name
- Age
- Address



Here, STUDENT is the entity type, and Student_ID, Student_Name, Age, and Address are its attributes.

🔹 3. Entity Set

An Entity Set is a collection of all individual entities of a particular entity type at a specific point in time.

- It contains actual instances of an entity type.
- Every entity in the set has the same attributes.
- Each entity is uniquely identified using a key attribute.
- The entity set may change when new entities are added or existing entities are deleted.

Example:

Consider the STUDENT entity set:

Student_ID	Student_Name	Age
101	Rahul	20
102	Priya	21
103	Amit	19
104	Sneha	20

Here, all four student records together form the STUDENT entity set.

🔹 4. Difference Between Entity Type and Entity Set

Entity Type	Entity Set
Defines a category of entities.	Contains actual entities.
Describes the structure.	Contains individual instances.
Represents common attributes.	Contains values of those attributes.

Generally remains fixed during database operation.	Changes when entities are inserted or deleted.
Example: STUDENT.	Example: Records of all students.

Q. Explain Specialization and Generalization. (6 Marks)

📘 1. Introduction

Specialization and Generalization are two important concepts in the Enhanced Entity-Relationship (EER) Model of DBMS.

They are used to organize entities into superclass and subclass relationships based on their common and different characteristics.

🔹 2. Specialization

Definition:

Specialization is a top-down approach in which a higher-level entity (superclass) is divided into smaller, more specific lower-level entities (subclasses) based on their distinct characteristics.

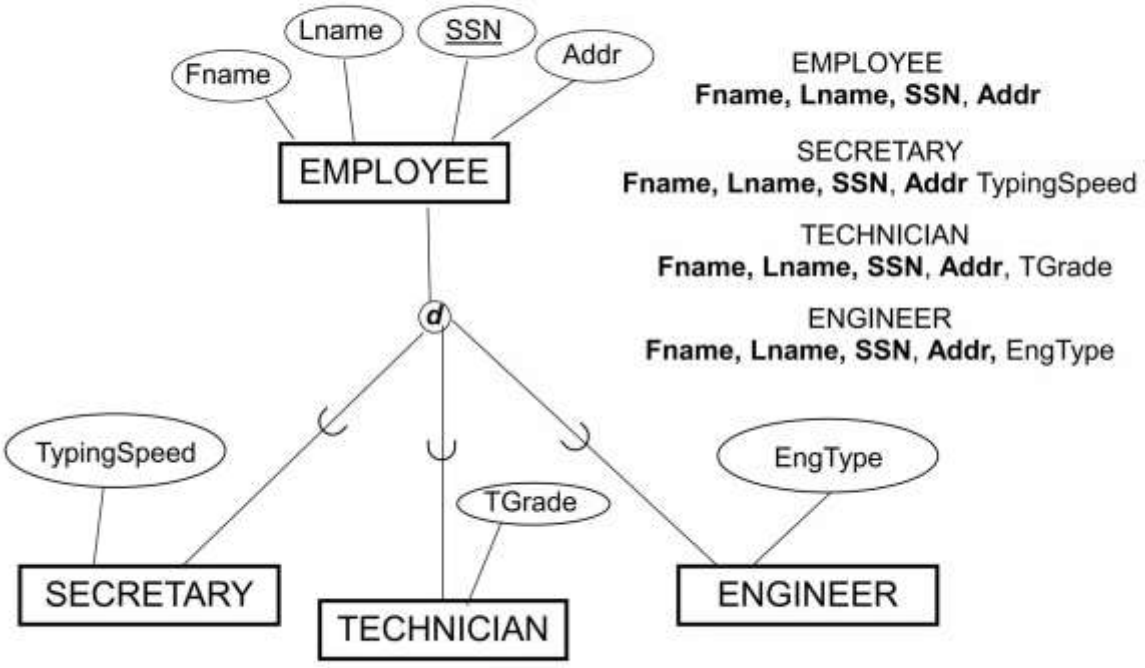
Characteristics:

- It divides one superclass into multiple subclasses.
- Each subclass has its own specific attributes.
- Subclasses inherit the common attributes of the superclass.
- It helps represent specific roles and properties of entities.

Example:

Consider an EMPLOYEE entity :

Example



Here:

- EMPLOYEE is the superclass.
- SECRETARY, TECHNICIAN and ENGINEER are subclasses.
- All three subclasses inherit common attributes such as Fname, Lname, SSN and Addr from EMPLOYEE.
- SECRETARY has an additional attribute called TypingSpeed.
- TECHNICIAN has an additional attribute called TGrade.
- ENGINEER has an additional attribute called EngType.
- The symbol 'd' represents a Disjoint constraint, which means an employee can belong to only one of these subclasses at a time.

🔹 3. Generalization

Definition:

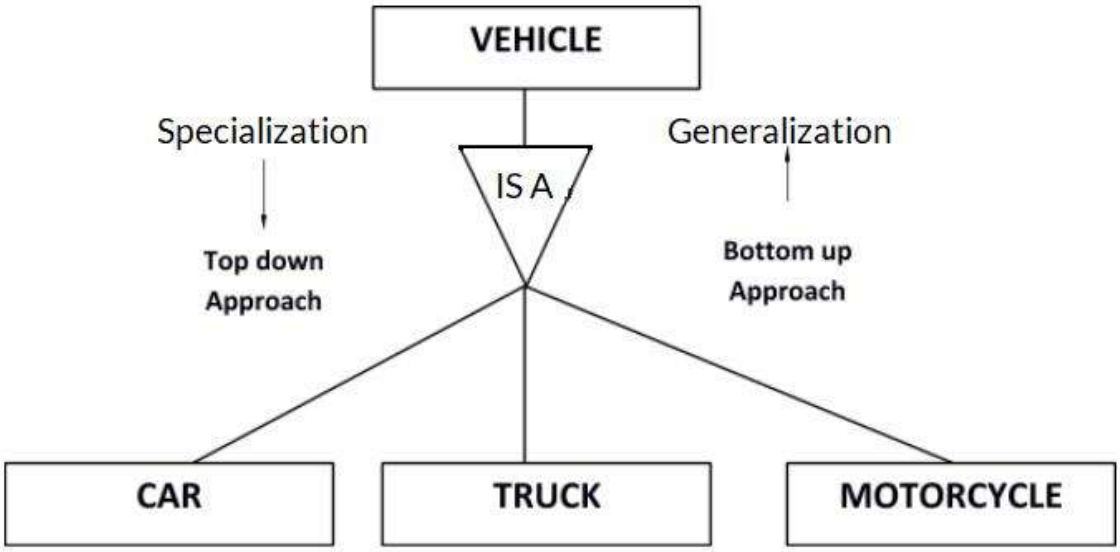
Generalization is a bottom-up approach in which two or more lower-level entity types are combined into a single higher-level entity (superclass) based on their common characteristics.

Characteristics:

- It combines multiple entity types into one superclass.
- It identifies common attributes among entities.
- The superclass contains common properties.
- It reduces data duplication and simplifies database design.

Example:

Consider CAR and BIKE entities that are combined into a common VEHICLE entity.



Here:

- VEHICLE is the superclass (higher-level entity).
- CAR, TRUCK and MOTORCYCLE are subclasses (lower-level entities).
- All three subclasses share common attributes such as Vehicle_ID, Model and Color from VEHICLE.

Theerthananda Gowda

- Specialization follows a top-down approach, where a superclass is divided into smaller subclasses.
- Generalization follows a bottom-up approach, where similar lower-level entities are combined into a common superclass.
- The IS-A relationship represents the relationship between the superclass and its subclasses.

🔹 4. Difference Between Specialization and Generalization

Specialization	**Generalization**
It follows a top-down approach.	It follows a bottom-up approach.
Divides one superclass into subclasses.	Combines multiple entity types into a superclass.
Focuses on specific characteristics.	Focuses on common characteristics.
Starts with a general entity.	Starts with specific entities.
Example: EMPLOYEE → MANAGER, ENGINEER.	Example: CAR, BIKE → VEHICLE.

🔹 5. Advantages

Specialization:

1. Helps represent specific characteristics.
2. Improves data organization.
3. Supports inheritance of common attributes.

Generalization:

1. Reduces data duplication.
2. Identifies common properties.
3. Simplifies database design.

✅ 6. Conclusion

Specialization and Generalization are important EER model concepts. Specialization follows a top-down approach by dividing a superclass into subclasses, whereas Generalization follows a bottom-up approach by combining similar entities into a common superclass.

Q. Explain the Main Phases of Database Design with a Diagram. (6 Marks)

📘 1. Introduction

Database Design is the process of planning and organizing data in a database so that it can be stored, accessed, updated, and managed efficiently.

The main aim of database design is to create a database that reduces data redundancy, maintains data consistency, and provides efficient data access.

🔹 2. Main Phases of Database Design

Database design consists of the following four main phases:

1. Requirements Collection and Analysis 📋

- Collects information about the data required by users.
- Identifies user needs, business rules, and database requirements.
- Determines what data should be stored and what operations should be performed.
- The output is a set of clearly defined database requirements.

Example: Collecting student details, course information, marks, and attendance requirements for a college database.

2. Conceptual Database Design 🎯

- Creates a high-level design of the database.
- Identifies entities, attributes, and relationships.
- Uses an Entity-Relationship (ER) diagram to represent the database structure.
- It is independent of any specific DBMS.

Example: Identifying STUDENT, COURSE, and FACULTY as entities.

Theerthananda Gowda

3. Logical Database Design 📊

- Converts the conceptual design into a logical database structure.
- Converts entities and relationships into tables.
- Defines primary keys, foreign keys, and constraints.
- Applies normalization to reduce data redundancy.

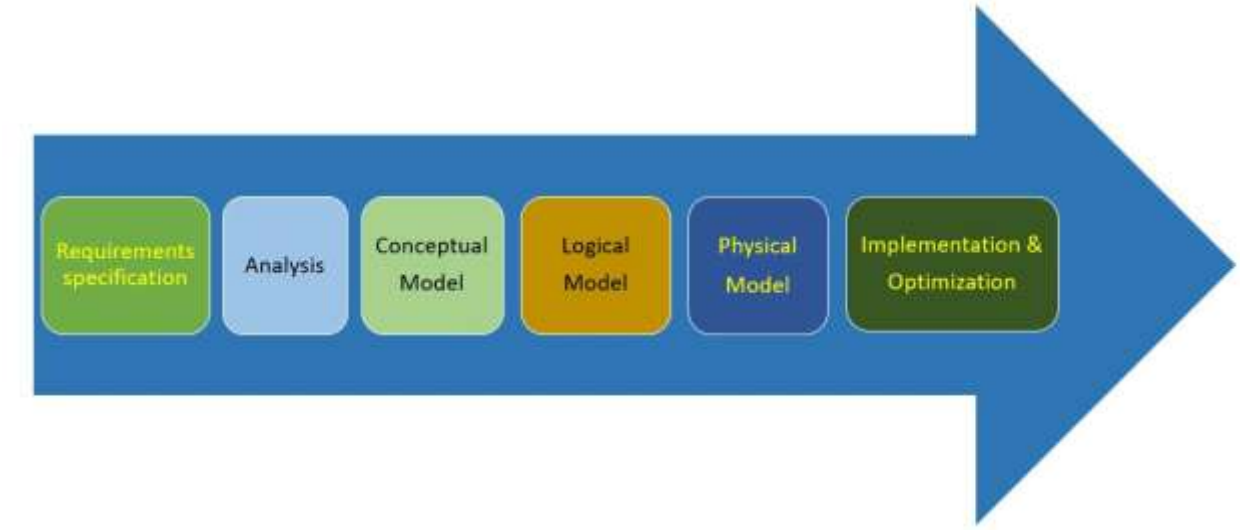
Example: Creating STUDENT, COURSE, and FACULTY tables with suitable keys.

4. Physical Database Design 💾

- Defines how the database will be physically stored in the computer system.
- Selects storage structures, file organization, and indexes.
- Focuses on improving database performance and storage efficiency.
- Depends on the selected DBMS.

Example: Creating indexes on Student_ID to speed up searching.

🔹 **3. Diagram of Main Phases of Database Design**



🔹 **4. Example**

Consider a College Management System:

1. Requirements: Collect student, teacher, and course details.

2. Conceptual Design: Create an ER diagram.
3. Logical Design: Convert the ER diagram into database tables.
4. Physical Design: Store tables and create indexes for faster access.

Q. Explain Cardinality Ratio with Examples. (6 Marks)

📘 1. Introduction

Cardinality Ratio is an important concept in the Entity-Relationship (ER) Model of DBMS. It defines the number of entities of one entity set that can be associated with the entities of another entity set through a relationship.

It helps determine how entities are connected in a database.

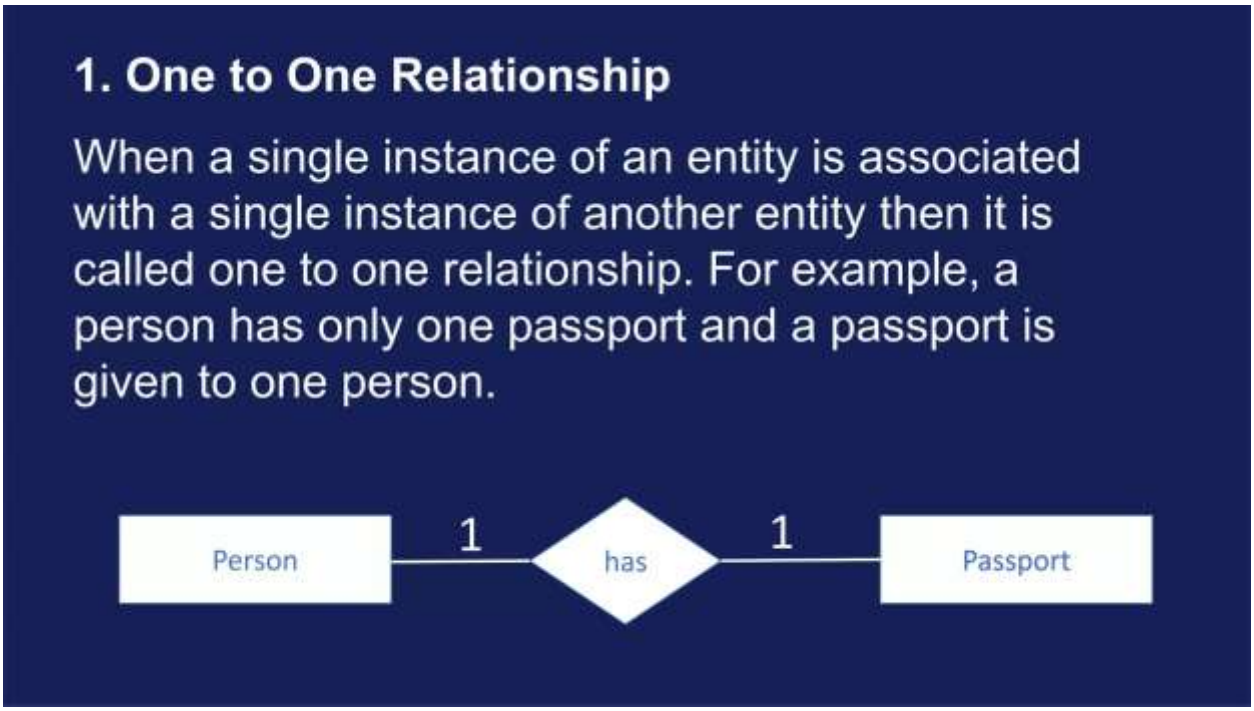
🔹 2. Types of Cardinality Ratios

There are four main types of cardinality ratios:

1. One-to-One (1:1) Relationship

- One entity of the first entity set is associated with only one entity of the second entity set.
- Similarly, one entity of the second entity set is associated with only one entity of the first entity set.

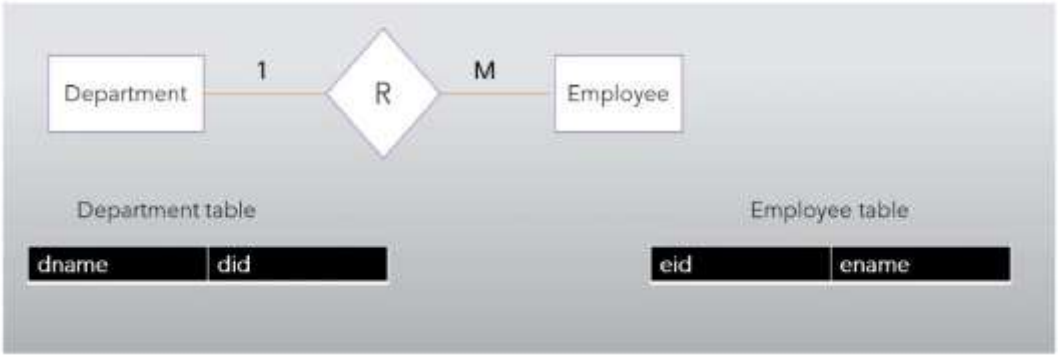
Example: One person has one passport, and one passport belongs to one person.



2. One-to-Many (1:N) Relationship

- One entity of the first entity set can be associated with many entities of the second entity set.
- Each entity of the second entity set is associated with only one entity of the first entity set.

Example: One department has many employees, but each employee belongs to one department.

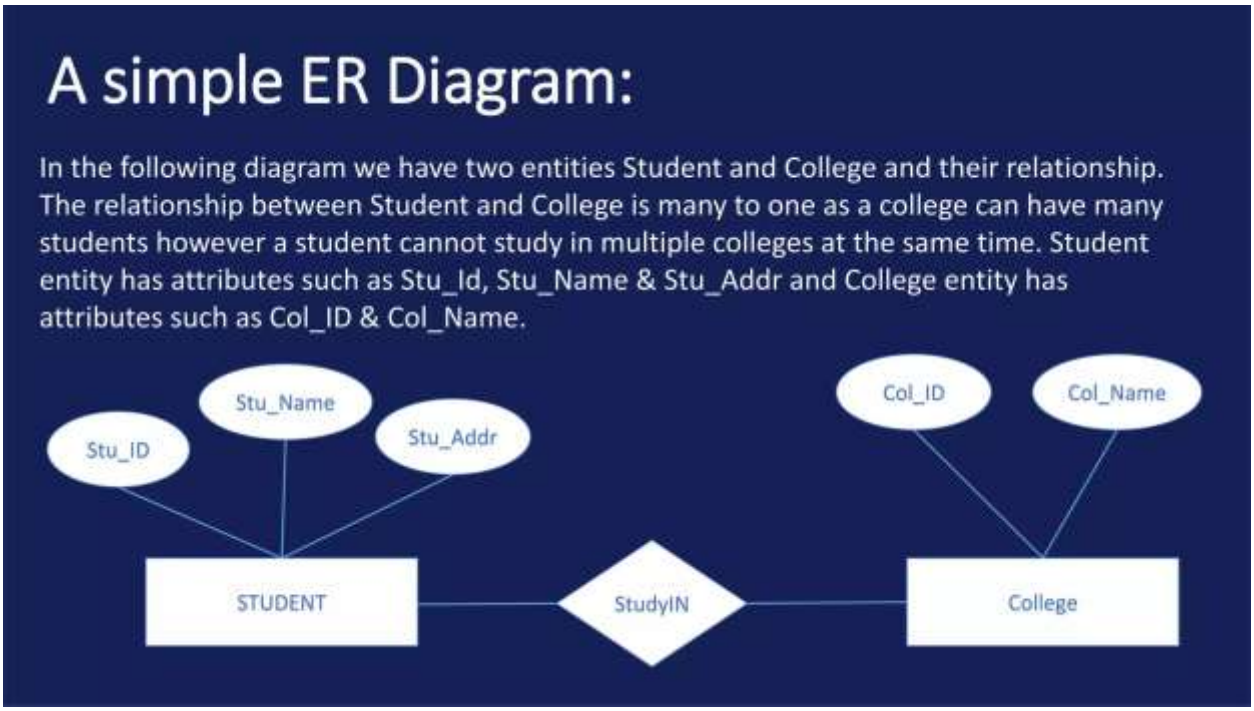


SCALER Topics

3. Many-to-One (N:1) Relationship

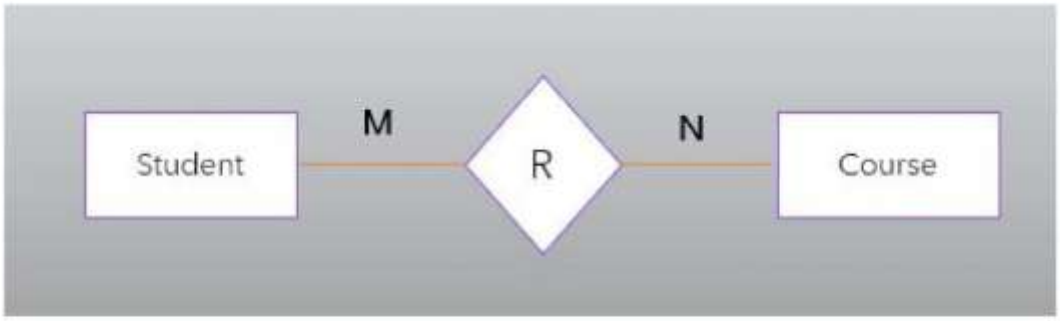
- Many entities of the first entity set can be associated with one entity of the second entity set.
- Each entity of the first entity set is associated with only one entity of the second entity set.

Example: Many students belong to one college.



4. Many-to-Many (M:N) Relationship

- Many entities of the first entity set can be associated with many entities of the second entity set.
- Each entity can have multiple related entities in the other entity set.Example: One student can enroll in many courses, and one course can have many students.



SCALER Topics

Theerthananda Gowda

🔹 3. Summary Table

Cardinality Ratio	Meaning	Example
One-to-One (1:1)	One entity to one entity	Person – Passport
One-to-Many (1:N)	One entity to many entities	Department – Employees
Many-to-One (N:1)	Many entities to one entity	Students – College
Many-to-Many (M:N)	Many entities to many entities	Students – Courses

🔹 4. Importance of Cardinality Ratio

1. Defines relationships between entities.
2. Helps in designing ER diagrams.
3. Helps identify database relationships.
4. Supports proper database table design.

Q. With a Diagram, Explain the Initial Conceptual Design of the Company Database. (7 Marks)

📘 1. Introduction

The Initial Conceptual Design of a Company Database is the process of identifying the main entities, attributes, and relationships required to represent the company's data.

It is usually represented using an Entity-Relationship (ER) Diagram.

The main purpose is to represent company information such as employees, departments, projects, and dependents before creating the actual database tables.

🔹 2. Main Entities of Company Database

The company database contains the following major entities:

1. EMPLOYEE 👨‍💼

- Stores employee information.
- Attributes: Employee_ID, Name, Address, Salary, Date_of_Birth.

Theerthananda Gowda

2. DEPARTMENT 🏢

- Stores department information.
- Attributes: Department_ID, Department_Name, Location.

3. PROJECT 📋

- Stores project details.
- Attributes: Project_ID, Project_Name, Project_Location.

4. DEPENDENT 👨‍👩‍👧

- Stores information about employees' dependents.
- Attributes: Dependent_Name, Relationship, Date_of_Birth.

5. DEPARTMENT_LOCATION 📍

- Stores the different locations of each department.

🔹 **3. Initial Conceptual Design Diagram of COMPANY Database**

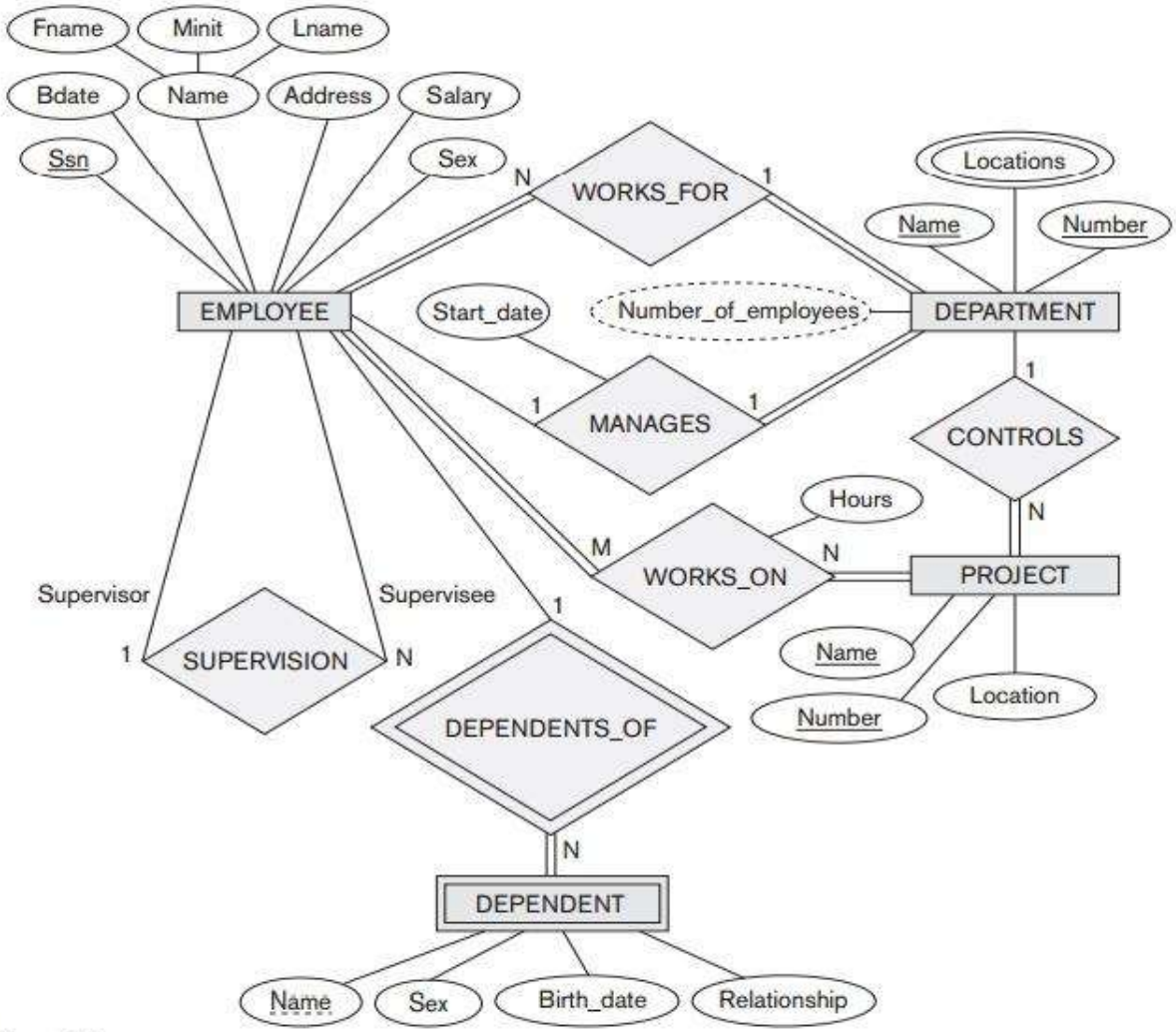


Figure 7.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 7.14.

- **4. Explanation of the Four Entity Types**

Entity Type	Important Attributes
DEPARTMENT	Name, Number, Locations, Manager, Manager_Start_Date
PROJECT	Name, Number, Location, Controlling_Department
EMPLOYEE	Name, SSN, Sex, Address, Salary, Birth_Date, Department, Supervisor

Theerthananda Gowda

DEPENDENT	Employee, Dependent_Name, Sex, Birth_Date, Relationship

Important points:

- DEPARTMENT: Name and Number are key attributes. Locations is a multivalued attribute.
- PROJECT: Name and Number are separate key attributes.
- EMPLOYEE: SSN is the key attribute. Name and Address may be composite attributes.
- DEPENDENT: Stores information about the dependents of employees.

🔹 5. Important Relationships in the Company Database

After identifying the initial entity types, relationships can be introduced:

1. WORKS_FOR: An employee works for a department.
2. MANAGES: An employee manages a department.
3. CONTROLS: A department controls projects.
4. WORKS_ON: Employees work on projects.
5. SUPERVISES: One employee supervises other employees.
6. DEPENDENTS_OF: An employee has dependents.

✅ 6. Conclusion

The initial conceptual design of the COMPANY database identifies four main entity types: EMPLOYEE, DEPARTMENT, PROJECT, and DEPENDENT. Their attributes and keys are identified using the ER model. This design provides the foundation for developing a complete database schema by introducing relationships and constraints.

Q. Compare and Contrast Specialization and Generalization. (6 Marks)

📘 1. Introduction

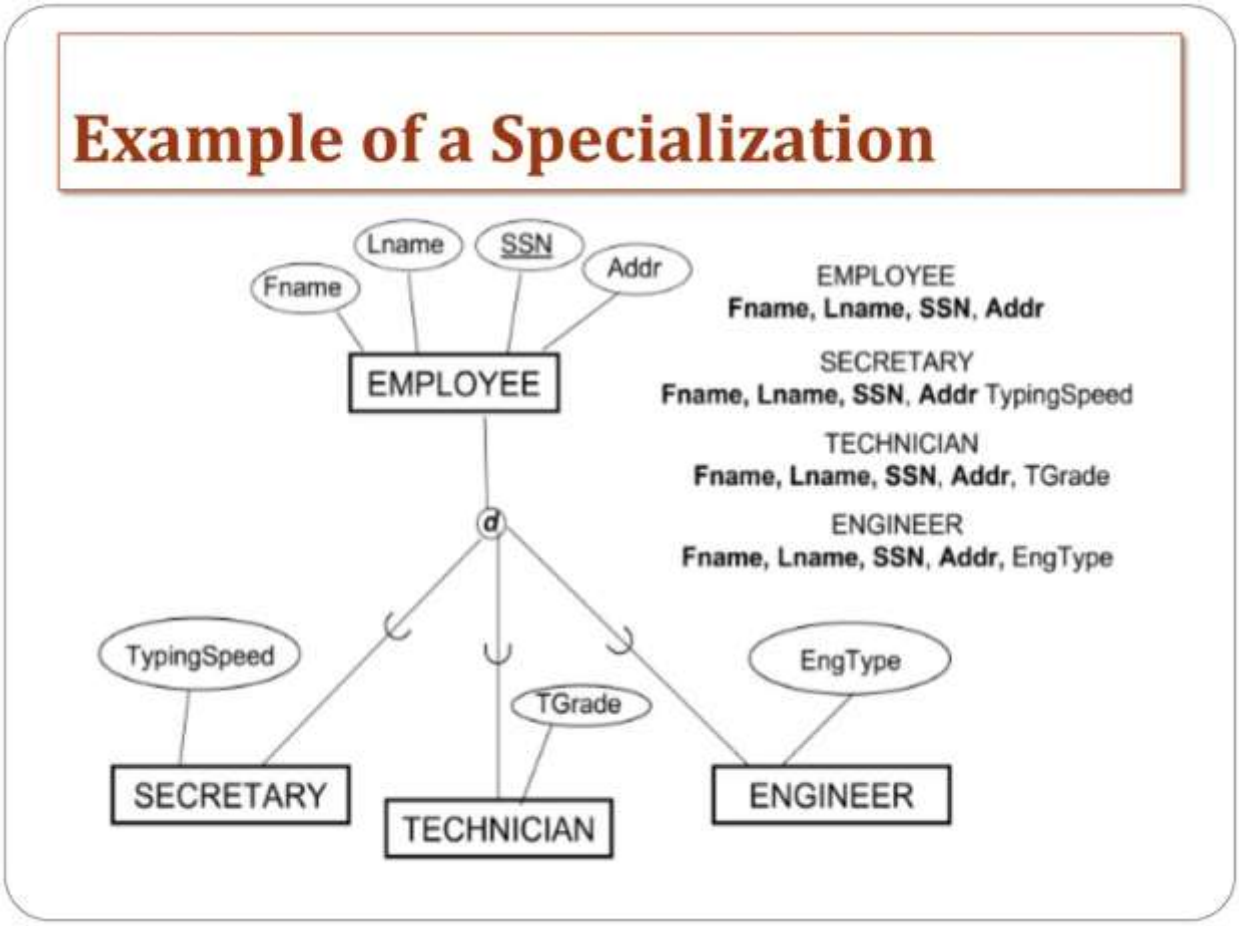
Specialization and Generalization are two important concepts of the Enhanced Entity-Relationship (EER) Model in DBMS.

Both are used to organize entities into superclass and subclass relationships based on their common and specific characteristics.

🔹 2. Specialization

Definition: Specialization is a top-down approach in which a higher-level entity (superclass) is divided into smaller, more specific entities (subclasses).

Example: An EMPLOYEE entity is divided into MANAGER and ENGINEER.



- EMPLOYEE is the superclass.
- MANAGER and ENGINEER are subclasses.

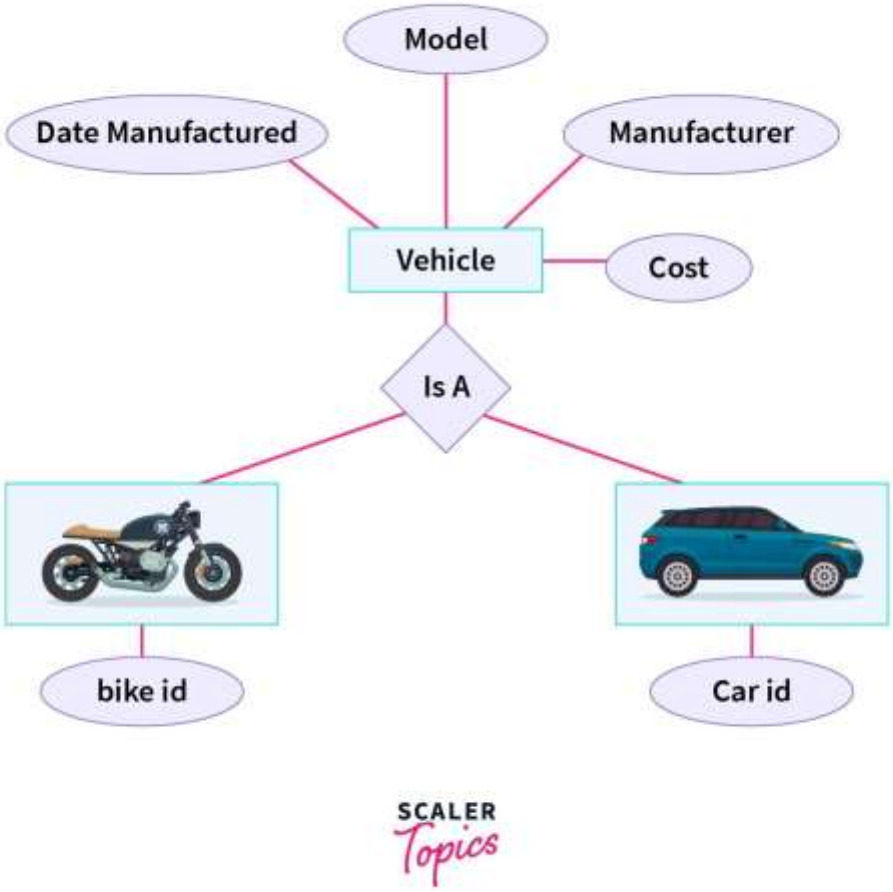
Theerthananda Gowda

- Subclasses inherit common attributes from the superclass.

🔹 3. Generalization

Definition: Generalization is a bottom-up approach in which two or more lower-level entities are combined into a single higher-level entity (superclass) based on common characteristics.

Example: CAR and BIKE entities are combined into a VEHICLE entity.



- CAR and BIKE are subclasses.
- VEHICLE is the superclass.
- Common attributes are combined into the superclass.

🔹 **4. Differences Between Specialization and Generalization**

Basis	Specialization	Generalization
1. Approach	Top-down approach.	Bottom-up approach.
2. Process	Divides one superclass into subclasses.	Combines multiple entities into one superclass.
3. Focus	Focuses on specific characteristics.	Focuses on common characteristics.
4. Starting Point	Starts with a general entity.	Starts with specific entities.
5. Purpose	Identifies different subgroups.	Identifies similarities among entities.
6. Result	Creates more specific entity types.	Creates a more general entity type.
7. Example	EMPLOYEE → MANAGER, ENGINEER.	CAR, BIKE → VEHICLE.

🔹 **5. Similarities**

1. Both are concepts of the EER model.
2. Both establish superclass-subclass relationships.
3. Both support inheritance of attributes.
4. Both help organize database entities effectively.

Q. Explain the Following:

📘 **(a) Relationship Types**

Definition:

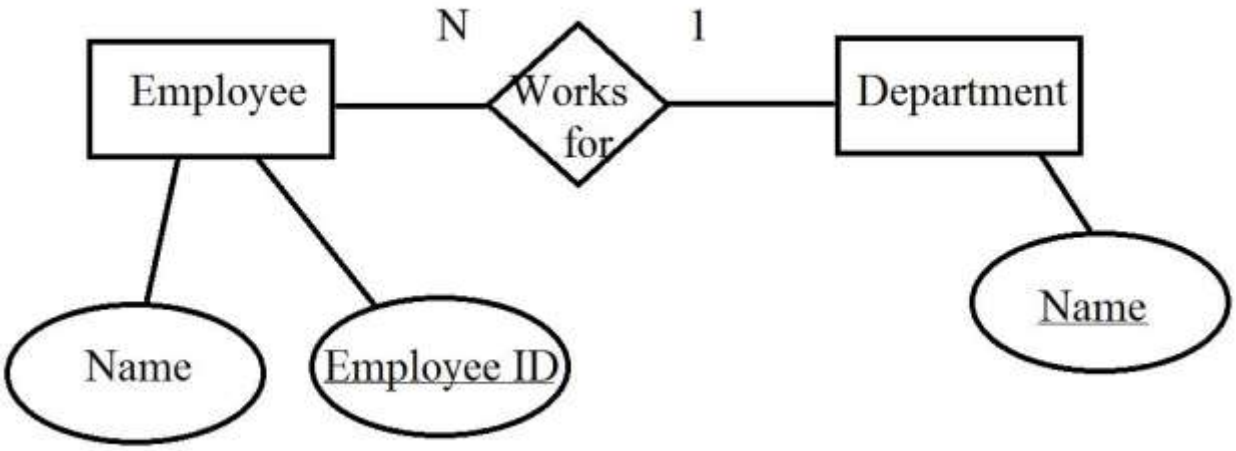
A Relationship Type defines the association between two or more entity types in an ER model.

- It represents how different entities are connected.
- It describes the relationship between entity types.
- It is represented by a diamond shape in an ER diagram.

Example:

An EMPLOYEE works for a DEPARTMENT.

Here, WORKS_FOR is the relationship type between EMPLOYEE and DEPARTMENT.



📘 **(b) Relationship Sets**

Definition:

A Relationship Set is a collection of all actual relationship instances of a particular relationship type at a specific point in time.

- It contains actual associations between entities.
- Each relationship instance connects specific entities.
- It may change when relationships are added or removed.

Example:

Consider the WORKS_FOR relationship:

Employee	Department
Rahul	HR
Priya	IT
Amit	Finance

These actual associations together form the WORKS_FOR relationship set.

📘 (c) Roles

Definition:

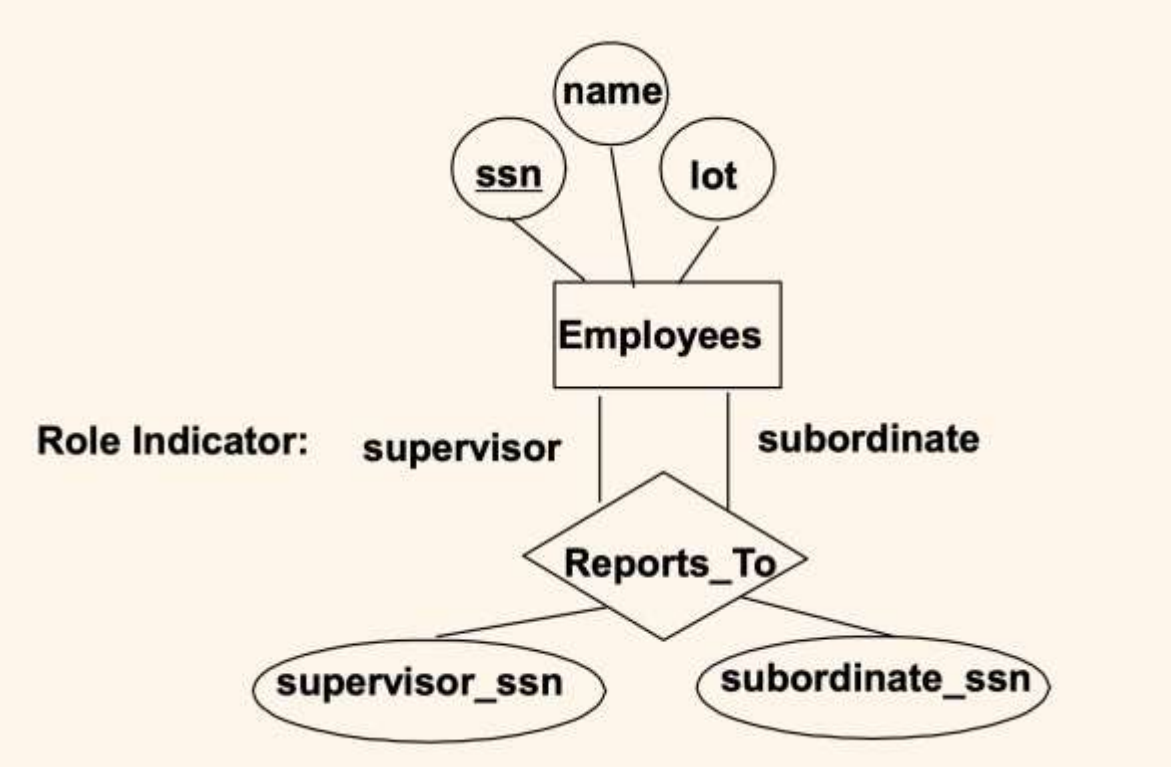
A Role specifies the function or purpose that an entity plays in a relationship.

- Roles help identify the participation of entities in a relationship.
- They are especially useful when the same entity type participates more than once in a relationship.
- Role names describe the meaning of each participation.

Example:

In an EMPLOYEE supervises EMPLOYEE relationship:

- One employee plays the role of Supervisor.
- Another employee plays the role of Subordinate.



📘 (d) Structural Constraints

Definition:

Structural Constraints define the restrictions on the number of relationships an entity can participate in within an ER model.

There are two main types of structural constraints:

1. Cardinality Ratio

- Specifies the maximum number of entities that can participate in a relationship.
- It defines four types:

Type	Meaning	Example
1:1	One-to-One	Person – Passport
1:N	One-to-Many	Department – Employees
N:1	Many-to-One	Students – College

M:N	Many-to-Many	Students – Courses

2. Participation Constraint

It specifies whether every entity must participate in a relationship or participation is optional.

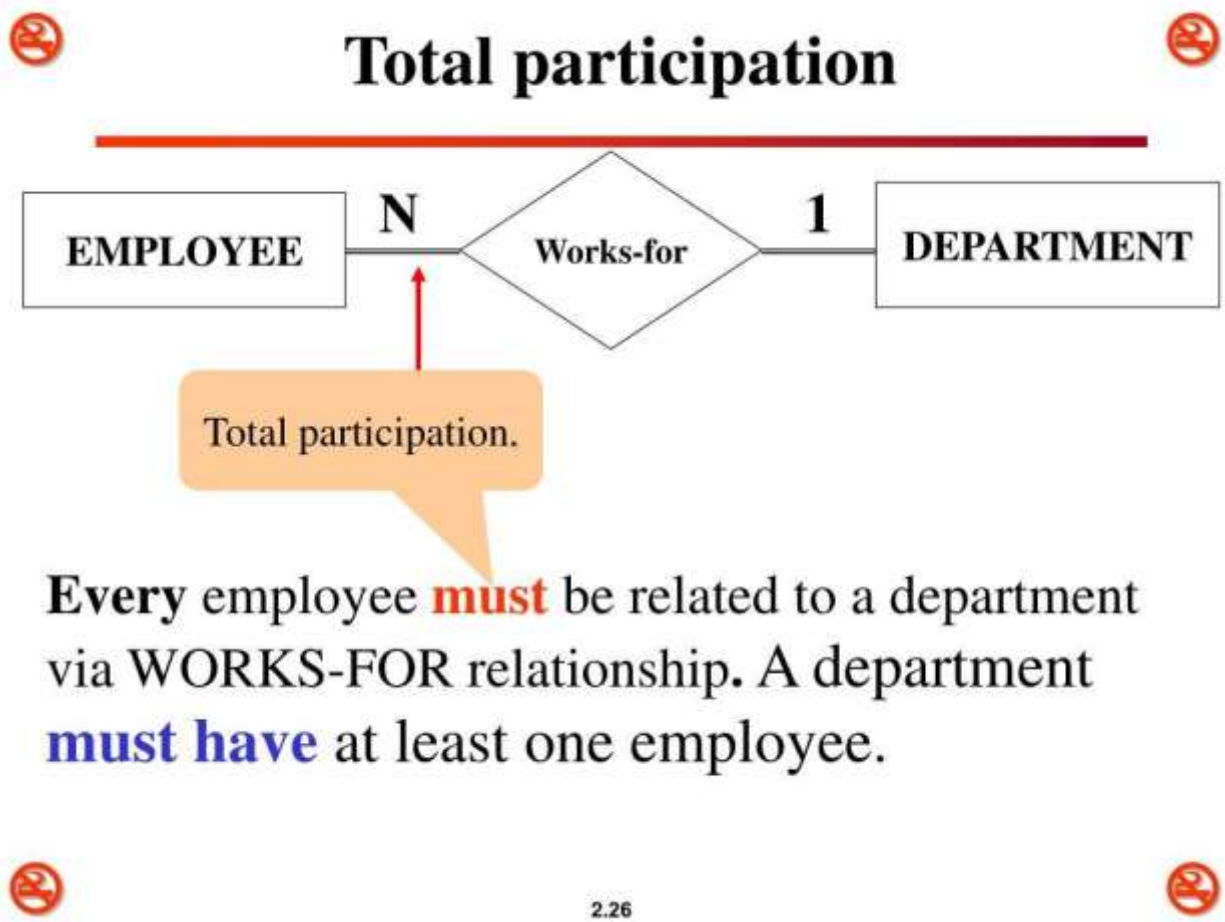
There are two types:

- Total Participation: Every entity must participate in the relationship. It is represented by a double line in an ER diagram.

Example: Every employee must belong to a department.

- Partial Participation: Only some entities participate in the relationship. It is represented by a single line in an ER diagram.

Example: Only some employees manage a department.



Theerthananda Gowda

Q4. Write a Note on the Following:

(a) Entity Types
(b) Entity Sets
(c) Attributes
(d) Keys

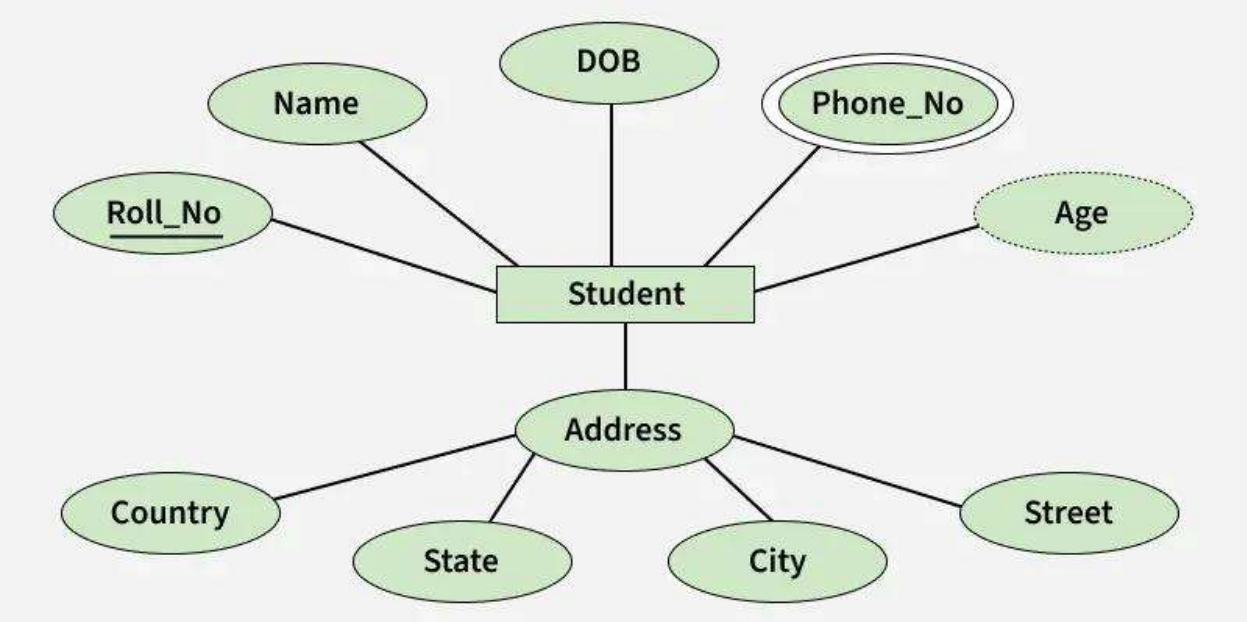
📘 (a) Entity Types

Definition:

An Entity Type is a collection or category of entities that have the same attributes and properties.

- It defines the structure of similar entities.
- Each entity type has a set of attributes.
- It is represented by a rectangle in an ER diagram.

Example: STUDENT is an entity type with attributes Student_ID, Name, Age, and Address.



📘 (b) Entity Sets

Definition:

An Entity Set is a collection of all individual entities belonging to a particular entity type at a specific point in time.

Theerthananda Gowda

- It contains actual instances of an entity type.
- Every entity has the same set of attributes.
- It changes when entities are inserted or deleted.

Example:

Consider the STUDENT entity set:

Student_ID	Name	Age
101	Rahul	20
102	Priya	21
103	Amit	19

These three student records together form the STUDENT entity set.

📘 (c) Attributes

Definition:

An Attribute is a property or characteristic that describes an entity.

- It provides information about an entity.
- Each entity can have one or more attributes.
- It is represented by an oval in an ER diagram.

Types of Attributes:

1. Simple Attribute: Cannot be divided further.

Example: Age, Salary.

2. Composite Attribute: Can be divided into smaller attributes.

Example: Name → First_Name, Middle_Name, Last_Name.

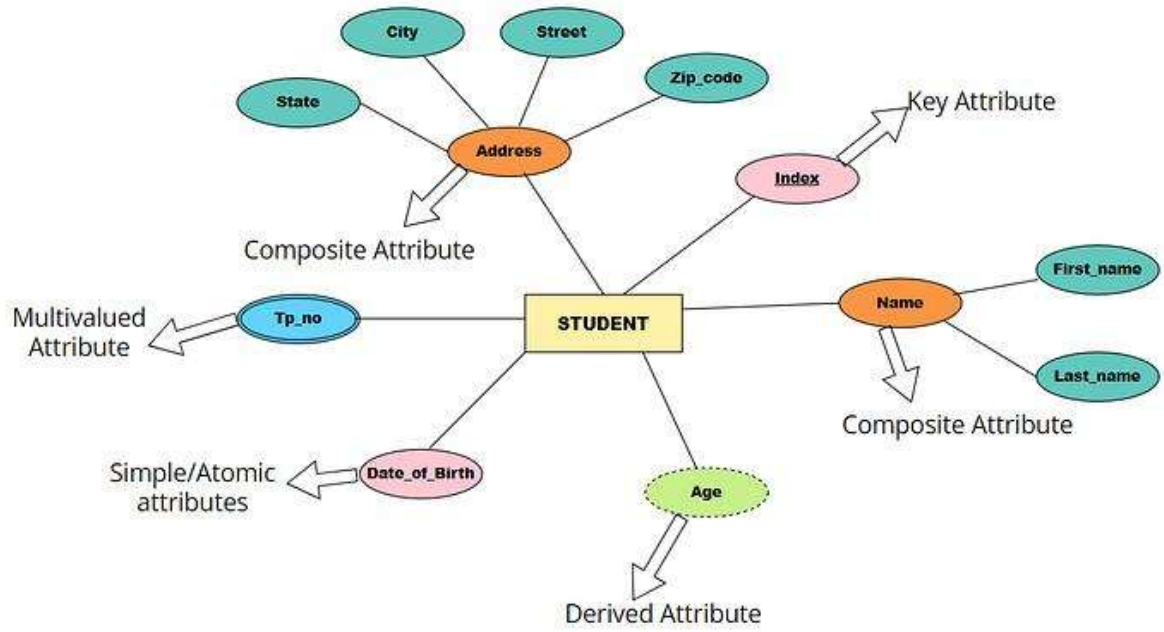
3. Single-Valued Attribute: Has only one value for each entity.

Example: Date_of_Birth.

4. Multi-Valued Attribute: Can have multiple values for an entity.

Example: Phone_Numbers.

5. Derived Attribute: Its value can be calculated from another attribute.



Example: Age calculated from Date_of_Birth.

📘 (d) Keys

Definition:

A Key is an attribute or combination of attributes used to uniquely identify an entity in a database.

Types of Keys:

1. Super Key

- A set of one or more attributes that uniquely identifies an entity.
- Example: Student_ID or Student_ID + Name.

2. Candidate Key

- A minimal super key with no unnecessary attributes.
- Example: Student_ID.

3. Primary Key

- A candidate key selected to uniquely identify each record.
- It cannot contain duplicate or NULL values.
- Example: Student_ID.

4. Alternate Key

- Candidate keys that are not selected as the primary key.
- Example: Email_ID, if Student_ID is the primary key.

5. Foreign Key

- An attribute that refers to the primary key of another table.
- It establishes relationships between tables.
- Example: Department_ID in the STUDENT table referencing the DEPARTMENT table.

🔹 **Example of Keys**

STUDENT Table

Student_ID (PK)	**Email_ID**	**Name**
101	a@gmail.com	Rahul
102	b@gmail.com	Priya
103	c@gmail.com	Amit

Here:

- Student_ID = Primary Key.
- Email_ID = Candidate Key (if unique).
- Student_ID + Name = Super Key.

Q. Explain Specialization and Generalization Hierarchies and Lattices with an Example. (7 Marks)

📘 1. Introduction

Specialization and Generalization are important concepts in the Enhanced Entity-Relationship (EER) Model of DBMS.

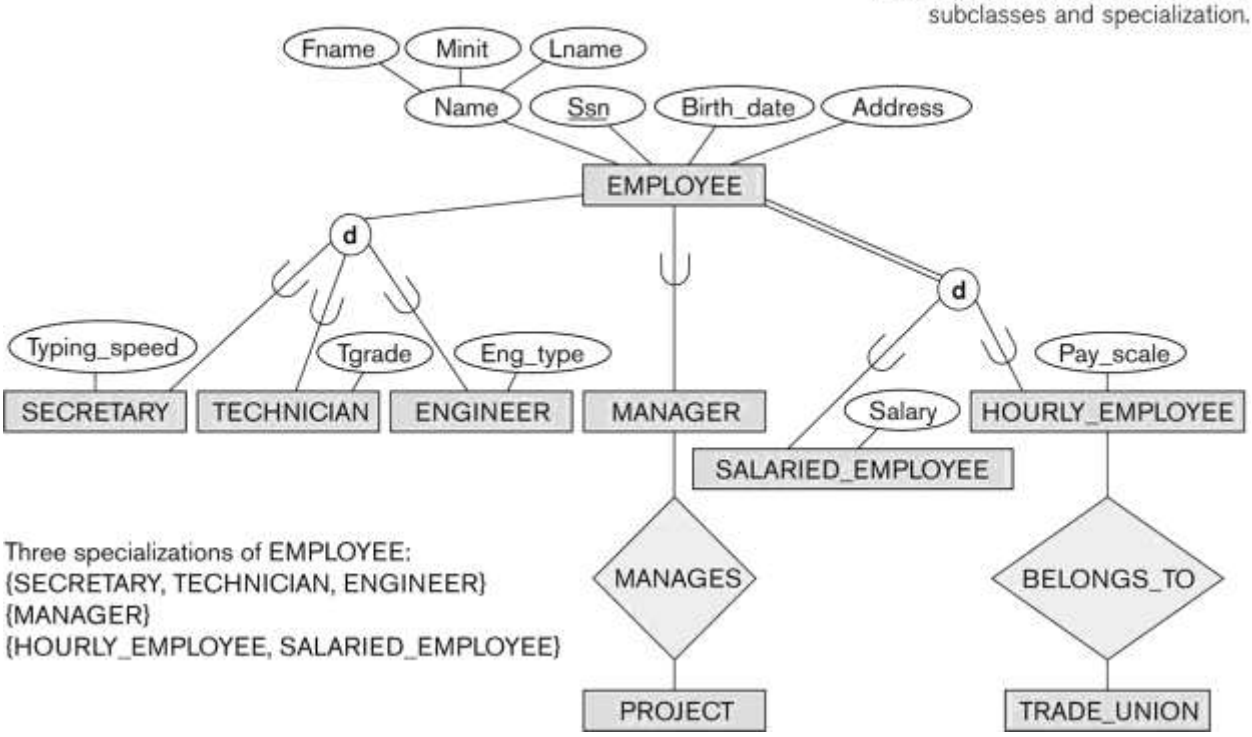
They are used to organize entities into superclass and subclass relationships based on their common and specific characteristics.

🔹 2. Specialization

Definition: Specialization is a top-down approach in which a higher-level entity (superclass) is divided into smaller, more specific entities (subclasses).

Example: An EMPLOYEE entity is divided into MANAGER and ENGINEER.

Figure 4.1
EER diagram notation to represent subclasses and specialization.



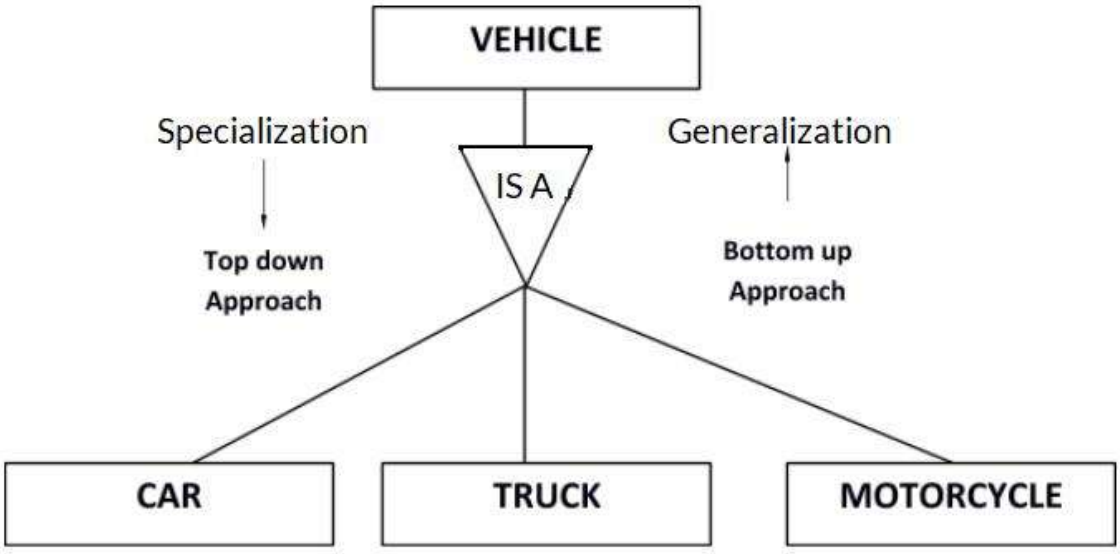
- EMPLOYEE is the superclass.
- MANAGER and ENGINEER are subclasses.
- Subclasses inherit the common attributes of EMPLOYEE.

Theerthananda Gowda

🔹 3. Generalization

Definition: Generalization is a bottom-up approach in which two or more lower-level entity types are combined into a common higher-level entity (superclass).

Example: CAR and BIKE are combined into a common VEHICLE entity.



- CAR and BIKE are subclasses.
- VEHICLE is the superclass.
- Common attributes such as Vehicle_ID, Model, and Color are placed in VEHICLE.

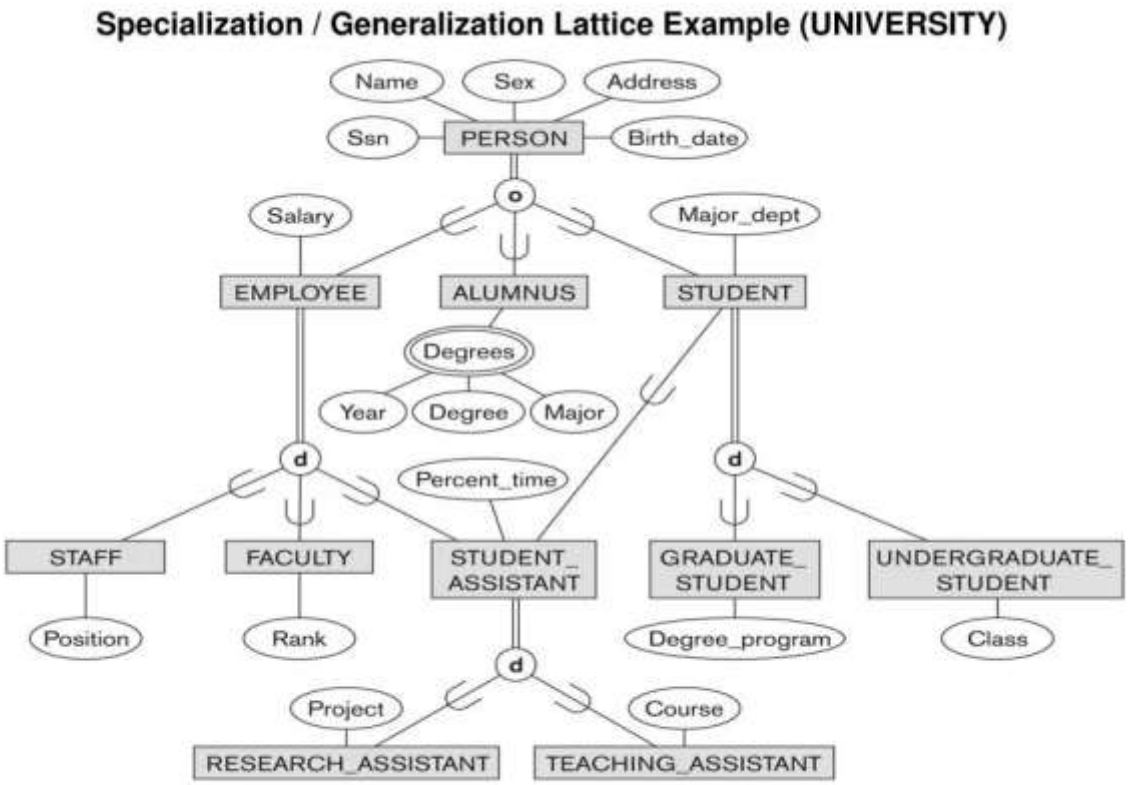
🔹 4. Specialization / Generalization Hierarchy 🌳

A Hierarchy is a structure in which each subclass has only one direct superclass.

- It follows a tree-like structure.
- A subclass inherits attributes from its parent superclass.
- It does not allow multiple direct superclasses for one subclass.

Example:

Consider a company database:



A specialization lattice with multiple inheritance for a UNIVERSITY database.

25

Here:

- PERSON is the main superclass.
- EMPLOYEE and STUDENT are subclasses of PERSON.
- MANAGER and ENGINEER are subclasses of EMPLOYEE.
- Each subclass has only one direct superclass.

🔹 **5. Specialization / Generalization Lattice** 🔗

A Lattice is a structure in which a subclass can have more than one direct superclass.

- It allows multiple inheritance.
- A subclass can inherit attributes from multiple superclasses.
- It provides more flexibility than a hierarchy.

Example:

Consider a university database:

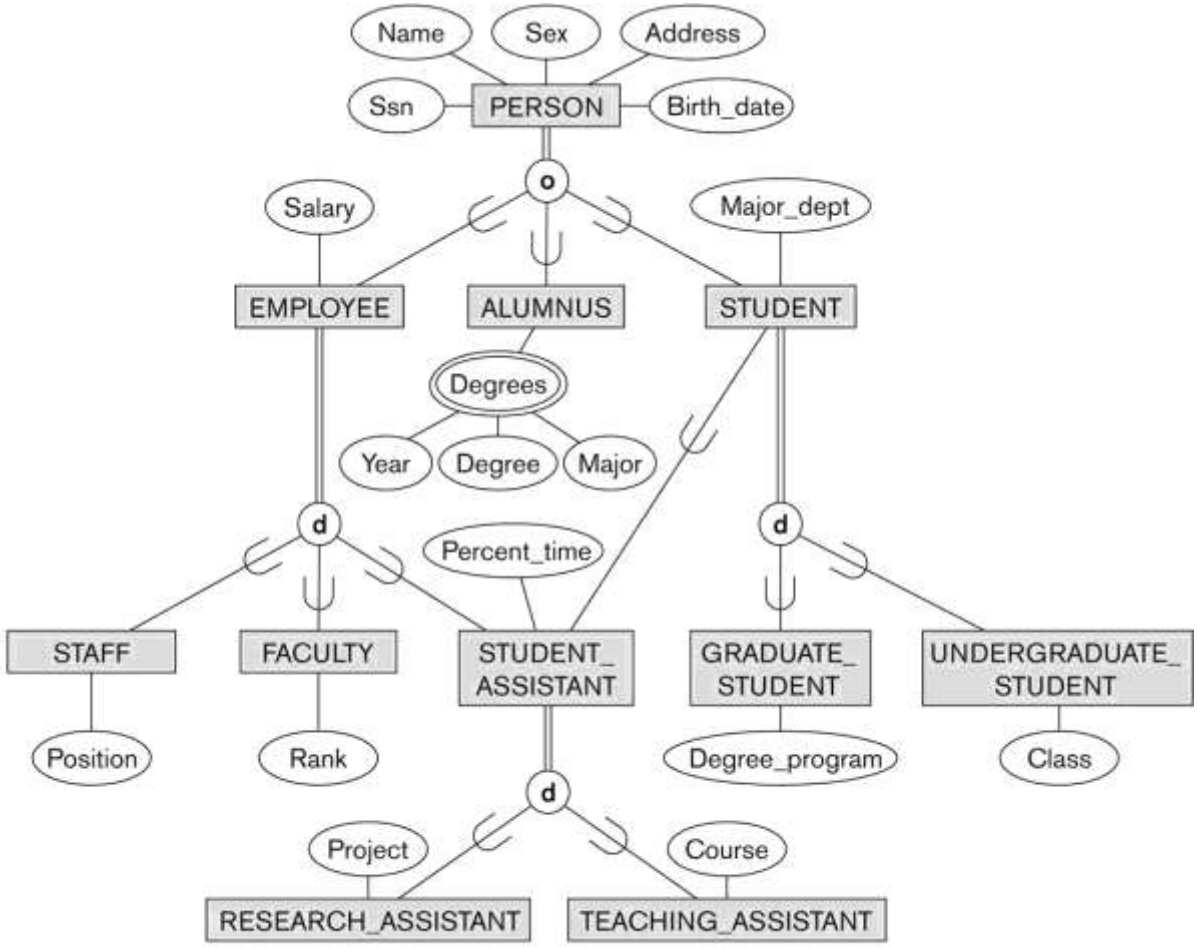


Figure 4.7
A specialization lattice with multiple inheritance for a UNIVERSITY database.

Here:

- PERSON is a common superclass.
- STUDENT and EMPLOYEE are subclasses of PERSON.
- TEACHING_ASSISTANT is a subclass of both STUDENT and EMPLOYEE.
- TEACHING_ASSISTANT inherits attributes from both superclasses.

🔹 **6. Difference Between Hierarchy and Lattice**

Hierarchy 🌳	**Lattice** 🔗
Each subclass has one direct superclass.	A subclass can have multiple direct superclasses.
Follows a tree-like structure.	Follows a network-like structure.
Does not support multiple inheritance.	Supports multiple inheritance.
Simple to understand and manage.	More flexible but complex.
Example: EMPLOYEE → MANAGER.	Example: STUDENT + EMPLOYEE → TEACHING_ASSISTANT.

Q. With an example, explain UML Class Diagram. (5 Marks)

1. Definition

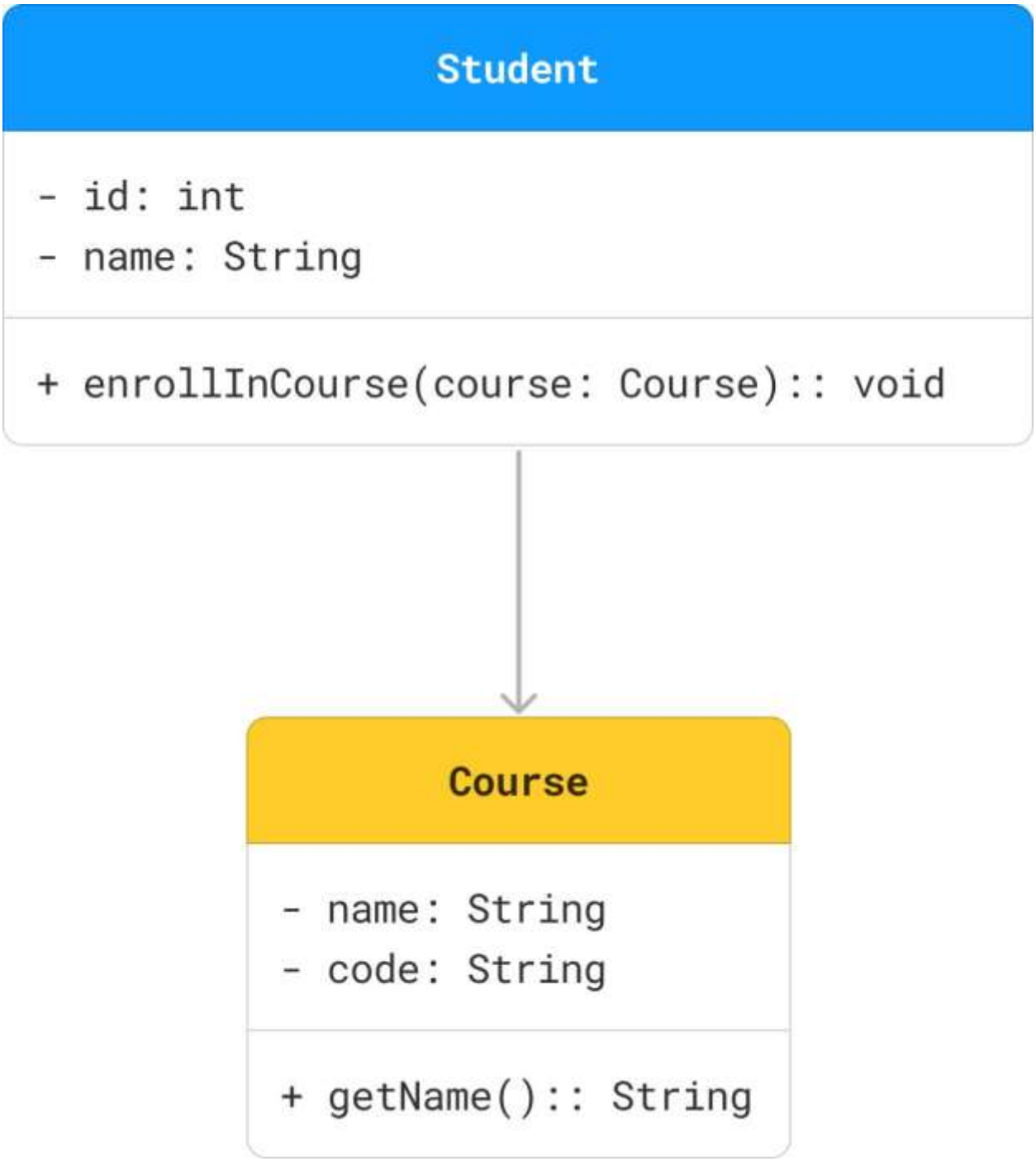
A UML Class Diagram is a structural diagram used to represent the classes, attributes, methods, and relationships between classes in a software system.

It describes the static structure of an object-oriented system.

2. Components of a Class Diagram

1. Class: Represents an object or entity in the system.
2. Attributes: Describe the properties or data members of a class.
3. Methods (Operations): Represent the functions or behaviours performed by a class.
4. Relationships: Show how different classes are connected.

3. UML Class Diagram Example – Student and Course



4. Explanation of the Example

The diagram represents a Student and Course management system.

- Student: Contains attributes such as ID and Name.

- Course: Contains attributes such as Name and Code.
- Methods: The Student class has an enrollInCourse() method to register for a course.
- Relationship: Student is connected to Course, showing that a student can enroll in a course.

Each class is represented using a rectangle divided into three sections:

- First section – Class Name.
- Second section – Attributes.
- Third section – Methods.

5. Advantages of UML Class Diagram

1. Helps to understand the structure of a system.
2. Makes software design easier.
3. Shows relationships between classes.
4. Helps developers during coding and maintenance.

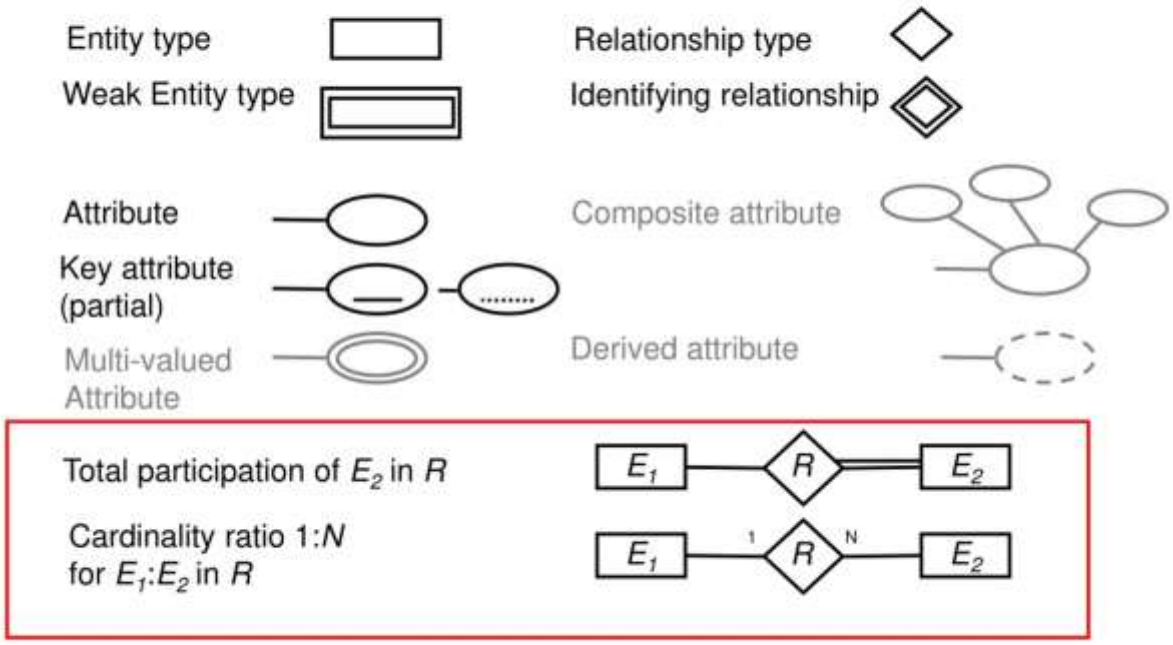
Q. Explain the notations used in an ER Diagram. (5 Marks)

1. Introduction

An Entity-Relationship (ER) Diagram is a graphical representation of entities, attributes, and relationships in a database. Different symbols are used to represent different components.

2. Notations used in an ER Diagram

ER Diagram notations



1. Entity (Rectangle)
   - Represents a real-world object or entity.
   - Example: Student, Employee.
2. Weak Entity (Double Rectangle)
   - An entity that cannot be uniquely identified by its own attributes.
   - Example: Dependent.
3. Attribute (Oval)
   - Represents the properties of an entity.
   - Example: Student_Name, Age.
4. Key Attribute (Underlined Oval)
   - Uniquely identifies each entity.

Theerthananda Gowda

- Example: Student_ID.

5. Multivalued Attribute (Double Oval)

- An attribute that can have multiple values.
- Example: Phone_Number.

6. Derived Attribute (Dashed Oval)

- An attribute whose value is calculated from another attribute.
- Example: Age calculated from Date_of_Birth.

7. Relationship (Diamond)

- Represents the association between two or more entities.
- Example: Student enrolls in Course.

8. Identifying Relationship (Double Diamond)

- Connects a weak entity with its owner entity.

3. Cardinality Notations

Cardinality represents the number of entities that can participate in a relationship.

- 1:1 – One-to-One
- 1:N – One-to-Many
- M:N – Many-to-Many

4. Conclusion

ER diagram notations help represent database entities, their attributes, and relationships clearly, making database design easier to understand.

Exam tip

For 5 marks, draw the symbols neatly and write their meanings with examples. Focus especially on Entity, Weak Entity, Attribute, Key Attribute, Multivalued Attribute, Derived Attribute, and Relationship.

Q. With an example, explain the ER Diagram. (5 Marks)

1. Definition

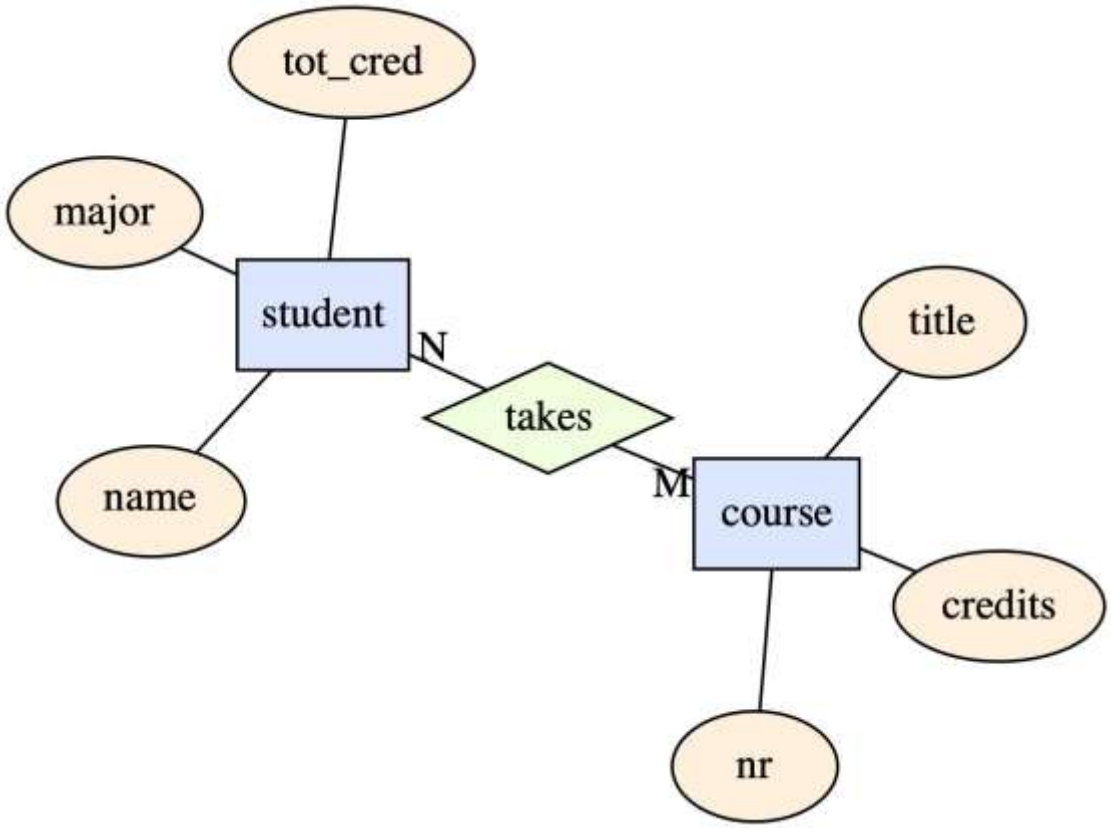
An Entity-Relationship (ER) Diagram is a graphical representation of entities, attributes, and relationships in a database.

It is used to design and understand the structure of a database before creating tables.

2. Components of an ER Diagram

1. Entity: A real-world object, represented by a rectangle. Example: Student.
2. Attribute: Properties of an entity, represented by an oval. Example: Student_ID, Name.
3. Relationship: Association between entities, represented by a diamond. Example: Enrolls.
4. Key Attribute: An attribute that uniquely identifies an entity. It is represented by an underlined name.

3. ER Diagram – Student and Course Example



Theerthananda Gowda

4. Explanation of the Example

Consider a college database containing Student and Course information.

- Student: Represents students and has attributes such as Name, Major and Total Credits.
- Course: Represents courses and contains attributes such as Course Number, Title and Credits.
- Takes: Represents the relationship between Student and Course.
- M:N Relationship: One student can take many courses, and one course can be taken by many students.

5. Advantages of ER Diagram

1. Helps in designing a database easily.
2. Clearly represents entities and relationships.
3. Reduces data duplication.
4. Makes database structure easy to understand.

Conclusion

ER diagrams provide a clear visual representation of a database and help developers design an efficient database structure.

Theerthananda Gowda